Delco

Banco de dados

Esquema MySQL 8.4: tabelas, relacionamentos, estados e consultas úteis



Copyright (©) 2026 Erik Pimenta

Versão 1.2.0 · Novembro de 2026 · Licença MIT

Sumário

1. Visão geral 3
Relacionamentos 3
Regras do esquema 4
2. Tabelas 5
users 5
api_tokens 5
audit_log 6
workflows 6
workflow_versions 7
workflow_runs 8
task_instances 9
task_attempts 10
task_logs 11
workers 11
integrations 12
integration_deliveries 13
3. Máquinas de estado 14
4. Consultas úteis 17
5. Preparar o banco 19
6. Evolução do esquema 20

Capítulo 1

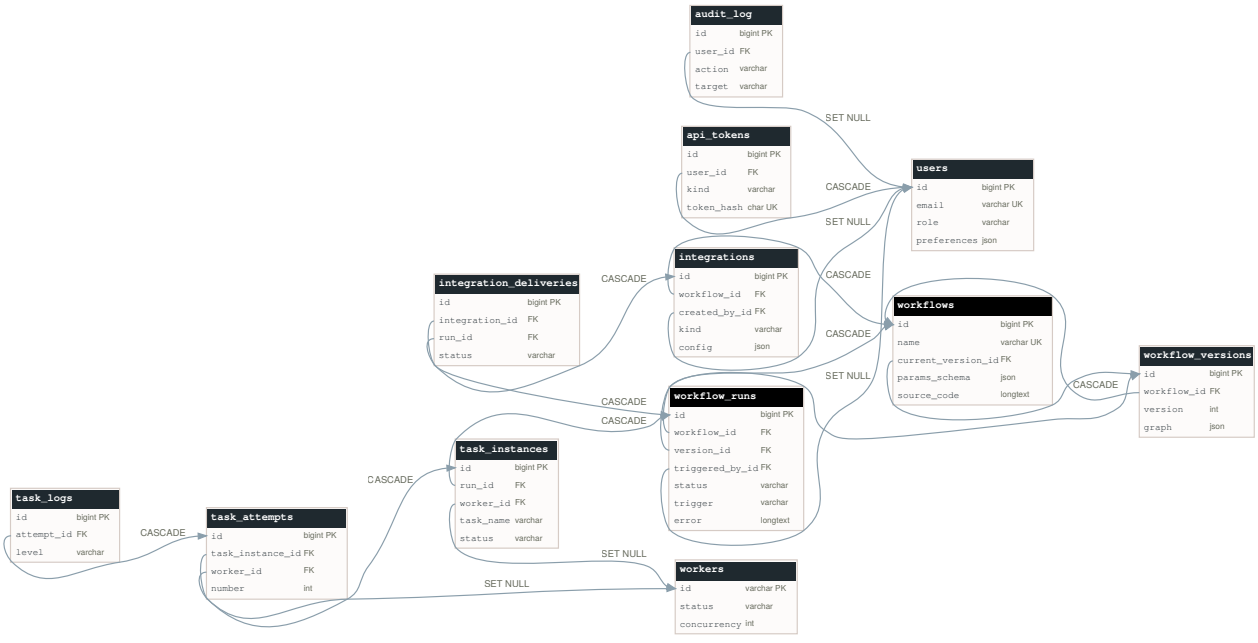
Visão geral

O MySQL é a fonte da verdade do Delco: definições de workflows, execuções, tarefas, tentativas, logs, workers, usuários, integrações e auditoria. O Redis só transporta mensagens, sinais de vida e contadores de limite; se for apagado, o agendador republica o que estava na fila e nada se perde.

São 12 tabelas em quatro áreas, criadas e evoluídas pelas migrações do Django (`delco/migrations`). A API aplica as migrações ao subir, com trava para que várias réplicas não migrem juntas.

Área	Tabelas	Conteúdo
Acesso	`users` · `api_tokens` · `audit_log`	Pessoas, preferências, sessões, tokens pessoais e o registro do que cada um fez
Definições	`workflows` · `workflow_versions`	Workflows lidos da pasta, parâmetros tipados, código e cada versão do grafo
Execução	`workflow_runs` · `task_instances` · `task_attempts` · `task_logs` · `workers`	Execuções, tarefas, tentativas, logs e processos que rodam o trabalho
Integrações	`integrations` · `integration_deliveries`	Destinos de aviso, gatilhos por URL e cada entrega com novas tentativas

Relacionamentos



Tabelas e chaves estrangeiras

Relação	Ao apagar o pai
`workflow_runs.workflow_id → workflows`	Apaga as execuções (CASCADE)

Relação	Ao apagar o pai
`workflow_runs.version_id → workflow_versions`	Sem ação: versões não são apagadas enquanto o workflow existe
`task_instances.run_id → workflow_runs`	Apaga as tarefas, tentativas e logs em cadeia
`task_attempts.task_instance_id → task_instances`	CASCADE
`task_logs.attempt_id → task_attempts`	CASCADE
`task_instances.worker_id` · `task_attempts.worker_id → workers`	Fica vazio (SET NULL)
`workflow_runs.triggered_by_id` · `audit_log.user_id` · `integrations.created_by_id → users`	SET NULL: o histórico fica
`api_tokens.user_id → users`	CASCADE
`integration_deliveries.integration_id → integrations`	CASCADE
`integration_deliveries.run_id → workflow_runs`	CASCADE
`integrations.workflow_id → workflows`	CASCADE (gatilho por URL)

Regras do esquema

- **InnoDB e utf8mb4** em todas as tabelas; datas em UTC (`datetime(6)`), convertidas para o fuso de cada pessoa no painel.
- **Compare-and-set**: toda mudança de estado inclui o estado esperado no `WHERE`; se outra máquina mudou antes, nada acontece.
- **Chaves únicas** garantem uma execução agendada por horário (`workflow_id` + `logical_date` para origem `schedule`), uma tarefa por nome em cada execução e uma tentativa por número.
- **FOR UPDATE SKIP LOCKED** nas leituras de trabalho do agendador e do worker, para que vários processos dividam a fila sem esperar uns pelos outros.
- **JSON nativo** para parâmetros, grafos, resultados, preferências e configuração de integrações.
- **Segredos**: tokens só como SHA-256; senhas com scrypt; URLs e tokens de bot nunca voltam inteiros pela API; parâmetros secretos mascarados na auditoria.
- **Limpeza**: execuções terminadas há mais de `DELCO_RETENTION_DAYS` (30) dias, entregas antigas, sessões vencidas e auditoria de mais de 180 dias saem sozinhas (ou pelo botão da tela Sistema).

Capítulo 2

Tabelas

As tabelas abaixo são lidas dos próprios modelos do Django ao gerar este documento, então tipos, nulos, padrões, índices e restrições sempre batem com o código. **PK** é a chave primária, **FK** uma chave estrangeira e **UK** um valor único. Campos marcados com (1.2.0) entraram nesta versão.

`users`

Acesso Pessoas que entram no painel e na API. Modelo `User`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`email`	varchar(254)	não			E-mail de login (único, sem diferenciar maiúsculas)
`name`	varchar(120)	não			Nome mostrado no painel e na auditoria
`password_hash`	varchar(255)	não			Senha com o hasher scrypt do Django
`role`	varchar(16)	não			admin, operator ou viewer
`is_active`	bool	não		true	Conta desativada não entra e perde os tokens
`failed_logins`	integer	não		0	Erros de senha seguidos
`locked_until`	datetime(6)	sim			Bloqueio temporário depois de muitos erros
`last_login_at`	datetime(6)	sim			Último login com sucesso
`preferences`	json	não		{}	Preferências do painel: tema, paleta, fuso, densidade, itens por página e avisos (1.2.0)
`created_at`	datetime(6)	não		now()	Criação
`updated_at`	datetime(6)	não		now()	Última mudança

Restrições

- `users_email_key` (UNIQUE): expressão
- `users_role_check` (CHECK): expressão
- `users_failed_logins_check` (CHECK): expressão

`api_tokens`

Acesso Sessões do painel e tokens pessoais; só o SHA-256 fica gravado. Modelo `ApiToken`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`user_id`	bigint	não	FK		Dono do token

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`name`	varchar(120)	não			Nome dado pelo usuário (ou Sessão no painel)
`kind`	varchar(16)	não			session ou personal
`token_hash`	varchar(64)	não	UK		SHA-256 do token (o valor nunca é gravado)
`prefix`	varchar(32)	não			Começo do token, para reconhecer na lista
`created_at`	datetime(6)	não		now()	Criação
`last_used_at`	datetime(6)	sim			Último uso (atualizado no máximo a cada minuto)
`expires_at`	datetime(6)	sim			Validade; vazio = sem validade
`revoked_at`	datetime(6)	sim			Quando foi revogado

Índices

- `api_tokens_user_idx`: user

Restrições

- `api_tokens_kind_check` (CHECK): expressão

`audit_log`

Acesso Registro de tudo que muda algo, guardado por 180 dias. Modelo `AuditLog`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`user_id`	bigint	sim	FK		Quem fez (vazio quando foi o próprio sistema)
`action`	varchar(64)	não			Ação, como run.trigger, run.backfill, run.mark ou worker.pause
`target`	varchar(255)	sim			Alvo, como run:118 ou workflow:ola_delco
`details`	json	não		{}	Detalhes da ação em JSON (parâmetros secretos mascarados)
`created_at`	datetime(6)	não		now()	Criação

Índices

- `audit_log_created_idx`: -created_at
- `audit_log_action_idx`: action, -id

`workflows`

Definições Um registro por workflow encontrado na pasta. Modelo `Workflow`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`name`	varchar(100)	não	UK		Nome do workflow (único)
`description`	longtext	não			Descrição do arquivo
`schedule`	varchar(120)	sim			Cron de 5 campos; vazio = só manual
`timezone`	varchar(64)	não		America/Sao_Paulo	Fuso da agenda
`max_active_runs`	integer	não		1	Execuções simultâneas permitidas
`default_params`	json	não		{}	Valores padrão dos parâmetros
`tags`	json	não		[]	Etiquetas
`source_file`	varchar(500)	sim			Arquivo relativo à pasta de workflows
`source_code`	longtext	sim			Código do arquivo na última leitura, mostrado no painel (1.2.0)
`source_hash`	varchar(64)	sim			SHA-256 do código (1.2.0)
`params_schema`	json	não		{}	Tipo, padrão, opções e limites de cada parâmetro (1.2.0)
`current_version_id`	bigint	sim	FK		Versão atual do grafo
`is_paused`	bool	não		false	Agenda pausada
`is_active`	bool	não		true	Falso quando o arquivo saiu da pasta
`next_run_at`	datetime(6)	sim			Próximo horário agendado
`created_at`	datetime(6)	não		now()	Criação
`updated_at`	datetime(6)	não		now()	Última mudança

Índices

- `workflows_due_idx`: next_run_at

Restrições

- `workflows_max_active_runs_check` (CHECK): expressão

`workflow_versions`

Definições Cada mudança no grafo vira uma versão; execuções antigas guardam a sua. Modelo `WorkflowVersion`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`workflow_id`	bigint	não	FK		Workflow dono da versão
`version`	integer	não			Número sequencial por workflow

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`fingerprint`	varchar(64)	não			SHA-256 do grafo (tarefas, dependências e opções)
`graph`	json	não			Grafo completo em JSON, inclusive run_timeout e retry_on quando usados
`source_code`	longtext	sim			Código do arquivo quando a versão nasceu (1.2.0)
`created_at`	datetime(6)	não		now()	Criação

Restrições

- `workflow_versions_version_key` (UNIQUE): workflow, version
- `workflow_versions_fingerprint_key` (UNIQUE): workflow, fingerprint
- `workflow_versions_version_check` (CHECK): expressão

`workflow_runs`

Execução Uma execução de um workflow. Modelo `WorkflowRun`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`workflow_id`	bigint	não	FK		Workflow
`version_id`	bigint	não	FK		Versão do grafo usada
`status`	varchar(16)	não		queued	queued, running, succeeded, failed ou cancelled
`trigger`	varchar(16)	não			schedule, manual, api, cli, webhook ou backfill (1.2.0)
`triggered_by_id`	bigint	sim	FK		Quem disparou
`logical_date`	datetime(6)	sim			Horário agendado que a execução representa
`params`	json	não		{}	Parâmetros já validados
`note`	longtext	sim			Observação (editável)
`error`	longtext	sim			Motivo da falha da execução, como o tempo limite (1.2.0)
`created_at`	datetime(6)	não		now()	Criação
`started_at`	datetime(6)	sim			Quando saiu da fila
`finished_at`	datetime(6)	sim			Quando terminou
`updated_at`	datetime(6)	não		now()	Última mudança

Índices

- `workflow_runs_workflow_idx`: workflow, -id
- `workflow_runs_open_idx`: status, id
- `workflow_runs_finished_idx`: finished_at

- `workflow_runs_logical_idx`: workflow, logical_date

Restrições

- `workflow_runs_schedule_key` (UNIQUE): expressão
- `workflow_runs_status_check` (CHECK): expressão
- `workflow_runs_trigger_check` (CHECK): expressão

`task_instances`

Execução Uma tarefa dentro de uma execução. Modelo `TaskInstance`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`run_id`	bigint	não	FK		Execução
`task_name`	varchar(100)	não			Nome da tarefa
`status`	varchar(16)	não		pending	pending, queued, running, retry_wait, succeeded, failed, skipped ou cancelled
`upstream`	json	não		[]	Tarefas das quais depende
`trigger_rule`	varchar(16)	não		all_success	all_success, all_done ou one_failed
`queue`	varchar(100)	não		default	Fila do Redis
`retries`	integer	não		2	Novas tentativas permitidas
`max_attempts`	integer	não		3	Total de tentativas permitidas
`retry_delay`	double precision	não		5	Espera base entre tentativas (s)
`retry_backoff`	double precision	não		2	Multiplicador da espera
`max_retry_delay`	double precision	não		600	Teto da espera (s)
`timeout_seconds`	double precision	sim			Tempo limite da tentativa
`attempts`	integer	não		0	Tentativas feitas
`worker_id`	varchar(200)	sim	FK		Worker com a posse da tarefa
`lease_expires_at`	datetime(6)	sim			Fim da posse; renovado a cada heartbeat
`queue_message_id`	varchar(64)	sim			ID da mensagem no Redis Stream
`cancel_requested`	bool	não		false	Pedido de parada pendente
`progress`	smallint	sim			Andamento de 0 a 100
`output`	json	sim			Resultado da tarefa (vai para ctx.upstream)

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`error`	longtext	sim			Erro ou motivo da última tentativa
`ready_at`	datetime(6)	sim			Quando a nova tentativa pode começar
`queued_at`	datetime(6)	sim			Quando foi para a fila
`started_at`	datetime(6)	sim			Início da primeira tentativa
`finished_at`	datetime(6)	sim			Fim
`updated_at`	datetime(6)	não		now()	Última mudança

Índices

- `task_instances_open_idx`: status, run
- `task_instances_lease_idx`: status, lease_expires_at
- `task_instances_finished_idx`: finished_at

Restrições

- `task_instances_run_task_key` (UNIQUE): run, task_name
- `task_instances_status_check` (CHECK): expressão
- `task_instances_trigger_rule_check` (CHECK): expressão
- `task_instances_retries_check` (CHECK): expressão
- `task_instances_max_attempts_check` (CHECK): expressão
- `task_instances_attempts_check` (CHECK): expressão
- `task_instances_progress_check` (CHECK): expressão

`task_attempts`

Execução Cada tentativa de uma tarefa. Modelo `TaskAttempt`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`task_instance_id`	bigint	não	FK		Tarefa
`number`	integer	não			Número da tentativa
`worker_id`	varchar(200)	sim	FK		Worker que rodou
`status`	varchar(16)	não		running	running, succeeded, failed, timed_out, lost, cancelled ou skipped (1.2.0)
`started_at`	datetime(6)	não		now()	Início
`finished_at`	datetime(6)	sim			Fim
`duration_ms`	integer	sim			Duração
`error`	longtext	sim			Erro resumido

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`traceback`	longtext	sim			Traceback completo do Python

Restrições

- `task_attempts_number_key` (UNIQUE): task_instance, number
- `task_attempts_number_check` (CHECK): expressão
- `task_attempts_status_check` (CHECK): expressão

`task_logs`

Execução Linhas de log de cada tentativa. Modelo `TaskLog`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`attempt_id`	bigint	não	FK		Tentativa
`logged_at`	datetime(6)	não		now()	Momento da linha
`level`	varchar(16)	não			debug, info, warning ou error
`message`	longtext	não			Texto (até 8000 caracteres)

Índices

- `task_logs_attempt_idx`: attempt, id

Restrições

- `task_logs_level_check` (CHECK): expressão

`workers`

Execução Processos worker conectados. Modelo `Worker`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	varchar(200)	não	PK		Nome do worker (host + pid ou DELCO_WORKER_NAME)
`hostname`	varchar(255)	não			Máquina
`pid`	integer	não			Processo
`queues`	json	não			Filas que consome
`concurrency`	integer	não			Vagas
`version`	varchar(32)	não			Versão do Delco
`status`	varchar(16)	não		online	online, paused (1.2.0), stopping ou offline
`started_at`	datetime(6)	não		now()	Início

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`last_heartbeat_at`	datetime(6)	não		now()	Último sinal
`stopped_at`	datetime(6)	sim			Quando parou

Restrições

- `workers_concurrency_check` (CHECK): expressão
- `workers_status_check` (CHECK): expressão

`integrations`

Integrações Destinos de aviso e gatilhos por URL. Modelo `Integration`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`name`	varchar(120)	não			Nome
`kind`	varchar(16)	não			slack, discord, teams, google_chat, telegram (1.2.0), email (1.2.0), webhook ou trigger
`url`	varchar(1000)	sim			URL do webhook (mascarada na API)
`secret`	varchar(128)	sim			Segredo do HMAC, token do bot do Telegram ou segredo da assinatura do gatilho
`token_hash`	varchar(64)	sim	UK		SHA-256 do token do gatilho por URL
`token_prefix`	varchar(16)	sim			Começo do token do gatilho
`workflow_id`	bigint	sim	FK		Workflow que o gatilho dispara
`events`	json	não		[]	Eventos assinados, inclusive run.recovered (1.2.0)
`workflows`	json	não		[]	Só estes workflows (vazio = todos)
`config`	json	não		{}	chat_id do Telegram, destinatários do e-mail e require_signature do gatilho (1.2.0)
`is_active`	bool	não		true	Ligada ou desligada
`created_by_id`	bigint	sim	FK		Quem criou
`created_at`	datetime(6)	não		now()	Criação
`updated_at`	datetime(6)	não		now()	Última mudança
`last_delivery_at`	datetime(6)	sim			Última entrega ou chamada
`last_status`	varchar(16)	sim			sent ou error

Índices

- `integrations_active_idx`: is_active

Restrições

- `integrations_kind_check` (CHECK): expressão

`integration_deliveries`

Integrações Cada aviso a enviar, com novas tentativas. Modelo `IntegrationDelivery`.

Campo	Tipo	Nulo	Chave	Padrão	Descrição
`id`	bigint	não	PK		Identificador
`integration_id`	bigint	não	FK		Integração
`run_id`	bigint	sim	FK		Execução do evento
`event`	varchar(32)	não			run.started, run.succeeded, run.failed, run.cancelled, run.recovered ou test
`status`	varchar(16)	não		pending	pending, sent ou failed
`attempts`	integer	não		0	Tentativas feitas
`next_attempt_at`	datetime(6)	não		now()	Próxima tentativa
`response_code`	smallint	sim			Código HTTP (250 no e-mail)
`error`	longtext	sim			Erro da última tentativa
`payload`	json	não			Corpo do evento em JSON
`created_at`	datetime(6)	não		now()	Criação
`sent_at`	datetime(6)	sim			Quando foi entregue

Índices

- `integration_deliveries_due_idx`: status, next_attempt_at
- `integration_deliveries_int_idx`: integration, -id

Restrições

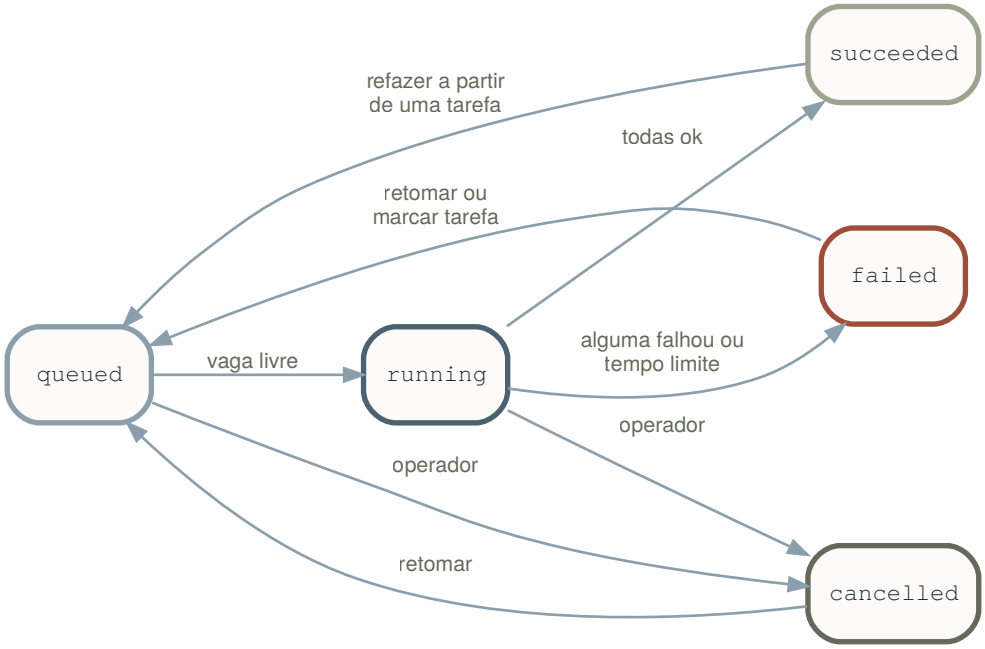
- `integration_deliveries_status_check` (CHECK): expressão
- `integration_deliveries_attempts_check` (CHECK): expressão

Capítulo 3

Máquinas de estado

Os estados vivem nas colunas `status`. Cada seta é uma atualização com compare-and-set; estados finais não mudam mais, exceto quando uma pessoa retoma a execução ou marca uma tarefa.

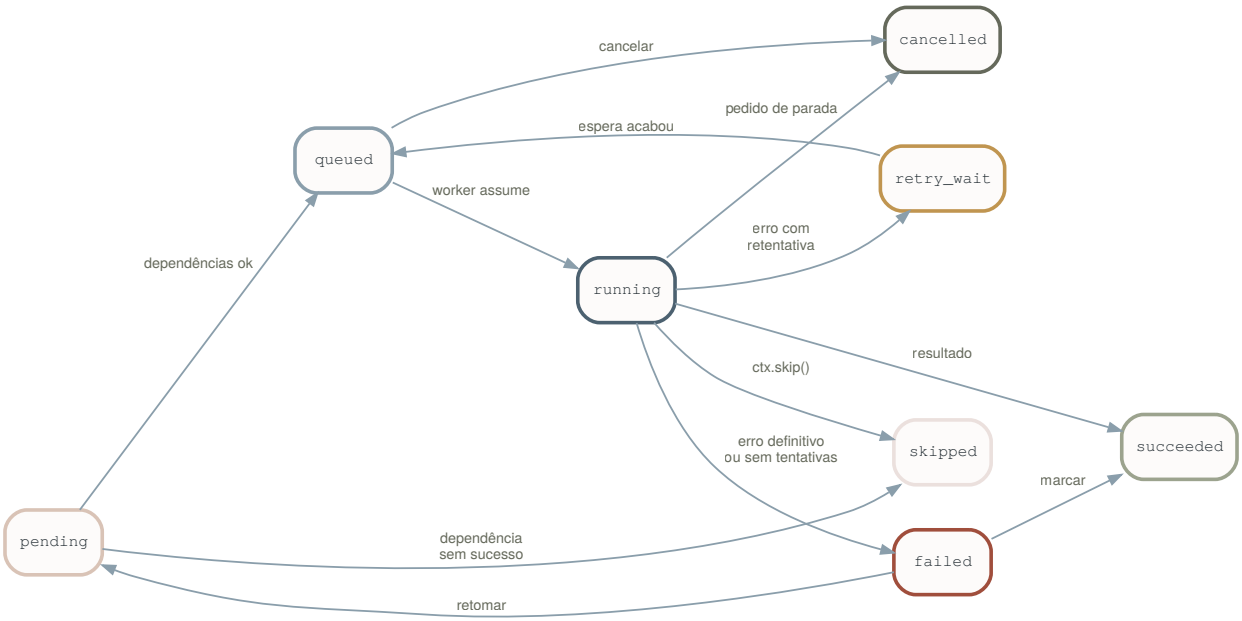
Execução (`workflow_runs.status`)



queued, running, succeeded, failed e cancelled

- Sai de `queued` quando há vaga dentro de `max_active_runs`.
- Termina em `succeeded` quando todas as tarefas acabaram sem falha (tarefas puladas contam como sucesso).
- Vai para `failed` se alguma tarefa falhou de vez ou se passou do `run_timeout`; nesse caso o motivo fica em `workflow_runs.error`.
- **Retomar da falha** e **marcar tarefa** devolvem uma execução terminada para `running`.

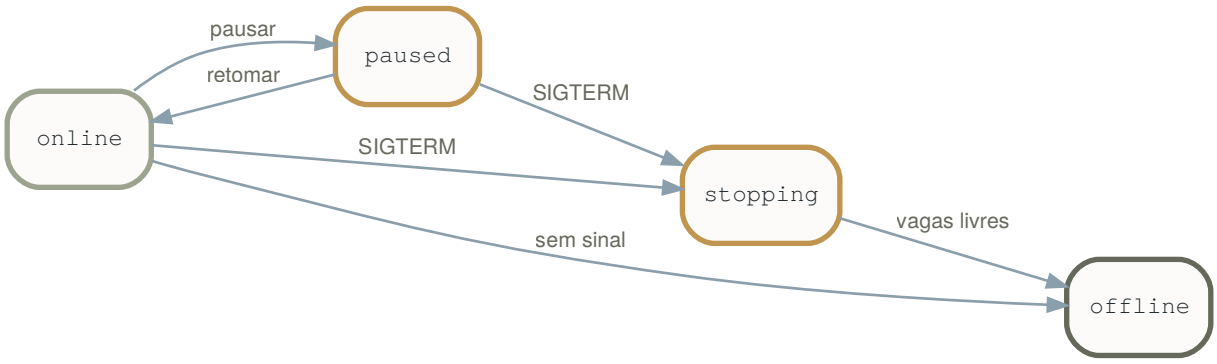
Tarefa (`task_instances.status`)



pending, queued, running, retry_wait, succeeded, failed, skipped e cancelled

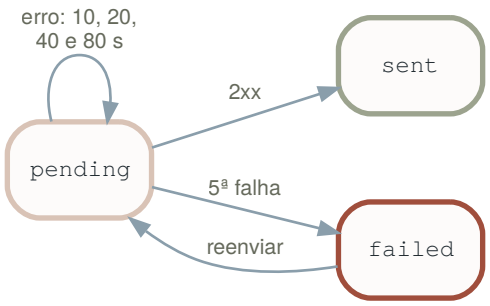
- `skipped` vem da regra de disparo, de `ctx.skip()` ou de uma marcação manual.
- `retry_wait` só acontece se o erro puder ser repetido (`retry_on` e `Fail(retry=True)`) e ainda houver tentativas.
- Lease vencido (`lease_expires_at`) faz a tarefa voltar a `retry_wait` com a tentativa marcada como `lost`.

Worker (`workers.status`)



online, paused, stopping e offline

Entrega (`integration_deliveries.status`)



pending, sent e failed

O estado `paused` do worker é novo na 1.2.0: o worker termina o que roda, devolve à fila o que já tinha lido e espera. Uma entrega `failed` pode voltar a `pending` pelo botão **Reenviar**.

Capítulo 4

Consultas úteis

Execuções em andamento por workflow

```
SELECT w.name, r.status, COUNT(*) AS total
FROM workflow_runs r JOIN workflows w ON w.id = r.workflow_id
WHERE r.status IN ('queued', 'running')
GROUP BY w.name, r.status
ORDER BY w.name;
```

Taxa de sucesso e duração dos últimos 7 dias

```
SELECT w.name,
       SUM(r.status = 'succeeded') / COUNT(*) AS taxa_sucesso,
       AVG(TIMESTAMPDIFF(SECOND, r.started_at, r.finished_at)) AS media_s
FROM workflow_runs r JOIN workflows w ON w.id = r.workflow_id
WHERE r.finished_at > UTC_TIMESTAMP() - INTERVAL 7 DAY
GROUP BY w.name
ORDER BY taxa_sucesso;
```

Tarefas que mais falham

```
SELECT w.name AS workflow, t.task_name, COUNT(*) AS falhas
FROM task_instances t
JOIN workflow_runs r ON r.id = t.run_id
JOIN workflows w ON w.id = r.workflow_id
WHERE t.status = 'failed' AND t.finished_at > UTC_TIMESTAMP() - INTERVAL 30 DAY
GROUP BY w.name, t.task_name
ORDER BY falhas DESC
LIMIT 10;
```

Execuções que estouraram o tempo limite

```
SELECT r.id, w.name, r.error, r.started_at, r.finished_at
FROM workflow_runs r JOIN workflows w ON w.id = r.workflow_id
WHERE r.status = 'failed' AND r.error IS NOT NULL
ORDER BY r.finished_at DESC
LIMIT 20;
```

Reprocessamentos de um período

```
SELECT r.id, r.logical_date, r.status, r.note
FROM workflow_runs r JOIN workflows w ON w.id = r.workflow_id
WHERE w.name = 'relatorio_vendas_diario' AND r.`trigger` = 'backfill'
ORDER BY r.logical_date;
```

Tarefas com posse vencida

```
SELECT id, run_id, task_name, worker_id, lease_expires_at
FROM task_instances
WHERE status = 'running' AND lease_expires_at < UTC_TIMESTAMP(6);
```

Workers e vagas ocupadas

```
SELECT wk.id, wk.status, wk.concurrency,
       (SELECT COUNT(*) FROM task_instances t WHERE t.worker_id = wk.id AND t.status = 'running')
AS ocupadas,
       wk.last_heartbeat_at
FROM workers wk
WHERE wk.status <> 'offline';
```

Parâmetros declarados de um workflow

```
SELECT name, JSON_PRETTY(params_schema) FROM workflows WHERE name = 'relatorio_vendas_diario';
```

Preferências das pessoas

```
SELECT email, preferences⟶»'$.theme' AS tema, preferences⟶»'$.palette' AS paleta,
       preferences⟶»'$.time_zone' AS fuso
FROM users WHERE is_active;
```

Entregas com falha nas últimas 24 horas

```
SELECT i.name, i.kind, d.event, d.attempts, d.response_code, d.error
FROM integration_deliveries d JOIN integrations i ON i.id = d.integration_id
WHERE d.status = 'failed' AND d.created_at > UTC_TIMESTAMP() - INTERVAL 1 DAY;
```

Quem fez o quê

```
SELECT a.created_at, u.email, a.action, a.target
FROM audit_log a LEFT JOIN users u ON u.id = a.user_id
WHERE a.action IN ('run.backfill', 'run.mark', 'run.delete', 'worker.pause')
ORDER BY a.created_at DESC
LIMIT 50;
```

> Prefira consultar pelo painel, pela API ou pela CLI: elas já aplicam permissões, mascaram segredos e registram ações na auditoria. Use SQL direto para relatórios e investigação, sem atualizar o estado à mão.

Capítulo 5

Preparar o banco

Com o Docker Compose nada precisa ser feito: o contêiner do MySQL cria o banco e o usuário a partir do `.env`. Num MySQL próprio (8.4 ou mais novo):

```
CREATE DATABASE delco CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE USER 'delco'@'%' IDENTIFIED BY 'troque-esta-senha';
GRANT ALL PRIVILEGES ON delco.* TO 'delco'@'%';
```

```
export DELCO_DATABASE_URL=mysql://delco:troque-esta-senha@db.interno:3306/delco
delco migrate
delco doctor
```

Recomendações

- Backup diário com `mysqldump --single-transaction --routines delco` ou snapshots do volume.
- O `delco doctor` e a tela Sistema conferem a versão do MySQL e se há migrações pendentes.
- Para testes, a suíte usa o banco `delco_test` (variável `DELCO_TEST_DATABASE_URL`), recriado a cada rodada.

Capítulo 6

Evolução do esquema

Migração	Versão	O que mudou
`0001_inicial`	1.0.0	Usuários, tokens, auditoria, workflows, versões, execuções, tarefas, tentativas, logs e workers, com chaves únicas e índices de trabalho
`0002_integracoes`	1.1.0	Integrações (Slack, Discord, Teams, Google Chat, webhook e gatilho por URL) e entregas com novas tentativas
`0003_versao_1_2`	1.2.0	`users.preferences`; `workflows.params_schema`, `source_code` e `source_hash`; `workflow_versions.source_code`; `workflow_runs.error`; `integrations.config`; origem `backfill`; estados `paused` (worker) e `skipped` (tentativa); tipos `telegram` e `email`; índices `audit_log_action_idx` e `workflow_runs_logical_idx`

As migrações só acrescentam colunas com padrão e índices, então rodam com o Delco no ar. Para voltar uma versão, use `python manage.py migrate delco 0002` antes de subir o código antigo.